For example, suppose you're writing an RSpec plugin and accompanying web service to help developers track long-term trends about their test suites. After each spec run, your plugin would report runtimes and pass/fail status to the service.

Your users will need some way to configure your plugin to use their assigned API keys for the web service. In your library, you'd call `add_setting` inside an `RSpec.configure` block, passing it the name of the setting you're creating:

09-configuring-rspec/05/rspec_configure.rb

```
RSpec.configure do |config|
  config.add_setting :spec_history_api_key
end
```

Once a developer has installed your plugin, he or she can set it up to use the API key like so:

09-configuring-rspec/05/rspec_configure.rb

```
RSpec.configure do |config|
  config.spec_history_api_key = 'a762bc901fga4b185b'
end
```

Even if you're just writing a library for your own projects, adding these kinds of configuration options can make it easier to use.

Your Turn

In this chapter, we looked at two ways to configure RSpec: the command-line options and the `configure` method. Command-line options are easy to discover, and they're great for one-off changes to RSpec's behavior. The `configure` method covers more of RSpec's behavior and gives you finer-grained control over how RSpec runs.

We're not asking anyone to memorize the entire set of configuration options. But the ones we've shown you here will help you turn RSpec into a comfortable, productive working environment.

Exercises

Now that you've seen what a wide array of configuration options RSpec offers, try your hand at using a few of them in these exercises.

Using a Custom Formatter

Find and read the documentation for the following RSpec formatters (some of which are more useful than others):